```
    },
    TableName: 'todo-task',
    IndexName: 'category-index',                              ②
    Limit: input['--limit']
  };
  if (input['--next'] !== null) {
    params.KeyConditionExpression += ' AND tid > :next';
    params.ExpressionAttributeValues[':next'] = {N: input['--next']};
  }
  if (input['--overdue'] === true) {
    params.FilterExpression = 'due < :yyyymmdd';              ③
    params.ExpressionAttributeValues[':yyyymmdd'] = {N: yyyymmdd};
  }
  [...]
  db.query(params, (err, data) => {
    if (err) {
      console.error('error', err);
    } else {
      console.log('tasks', data.Items.map(mapTaskItem));
      if (data.LastEvaluatedKey !== undefined) {
        console.log('more tasks available with --next='
          + data.LastEvaluatedKey.tid.N);
      }
    }
  });
}
```

① A query against an index works the same as a query against a table ...

② ... but you must specify the index you want to use.

③ Filtering works the same as with tables.

When following our example, use the following command to fetch John and Emma’s shopping tasks:

```
node index.js task-la shopping
```

But you’ll still have situations where a query doesn’t work. For example, you can’t retrieve all users from `todo-user` with a query. Therefore, let’s look at how to scan through all items of a table.